

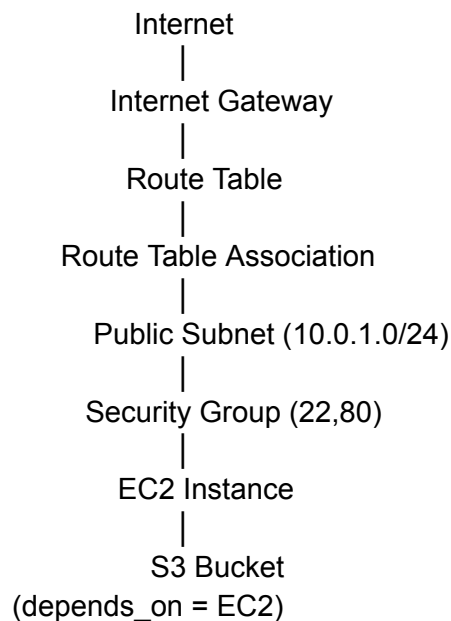
Day 62 - Providers, Resources and Dependencies

Objective

By the end of this lab you will create:

- ☒ VPC
 - ☒ Public Subnet
 - ☒ Internet Gateway
 - ☒ Route Table
 - ☒ Route Table Association
 - ☒ Security Group
 - ☒ EC2 Instance
 - ☒ S3 Bucket
 - ☒ Dependency Graph
 - ☒ Destroy everything
-

Architecture



Step 1 - Login to EC2

```
ssh -i key.pem ubuntu@<Public-IP>
```

Go to Terraform folder.

```
cd ~/terraform
```

Step 2 - Create Project

```
mkdir terraform-aws-infra  
cd terraform-aws-infra
```

Check

```
pwd
```

Should show

```
~/terraform/terraform-aws-infra
```

Step 3 - Create Files

```
touch providers.tf  
touch main.tf
```

Check

```
ls
```

Output

```
main.tf  
providers.tf
```

Step 4 - Write providers.tf

Open

```
nano providers.tf
```

Paste

```
terraform {
```

```
required_providers {  
  
  aws = {  
  
    source = "hashicorp/aws"  
  
    version = "~> 5.0"  
  
  }  
  
}  
  
provider "aws" {  
  
  region = "us-east-1"  
  
}
```

Save

CTRL+O

ENTER

CTRL+X

Step 5 - Initialize Terraform

terraform init

Example output

Installing hashicorp/aws v5.xx.x

Terraform has been successfully initialized!

Notice the version.

Example

5.89.0

```
ubuntu@ip-172-31-79-174:~/terraform-aws-infra$ terraform init
Initializing the backend...

Initializing provider plugins...
- Finding hashicorp/aws versions matching "~> 5.0"...
- Installing hashicorp/aws v5.100.0...
- Installed hashicorp/aws v5.100.0 (signed by HashiCorp)

Terraform has created a lock file .terraform.lock.hcl to record the provider
selections it made above. Include this file in your version control repository
so that Terraform can guarantee to make the same selections by default when
you run "terraform init" in the future.

Terraform has been successfully initialized!

You may now begin working with Terraform. Try running "terraform plan" to see
any changes that are required for your infrastructure. All Terraform commands
should now work.

If you ever set or change modules or backend configuration for Terraform,
rerun this command to reinitialize your working directory. If you forget, other
commands will detect it and remind you to do so if necessary.
ubuntu@ip-172-31-79-174:~/terraform-aws-infra$
```

Step 6 - Check Lock File

ls -la

You'll see

.terraform/

.terraform.lock.hcl

Open

cat .terraform.lock.hcl

You'll see

provider "registry.terraform.io/hashicorp/aws"

This file locks the provider version so everyone uses the same version.

```

ubuntu@ip-172-31-79-174:~/terraform-aws-infra$ cat .terraform.lock.hcl
# This file is maintained automatically by "terraform init".
# Manual edits may be lost in future updates.

provider "registry.terraform.io/hashicorp/aws" {
  version      = "5.100.0"
  constraints = "~> 5.0"
  hashes = [
    "h1:edXOJWE4ORX8Fm+dpVpICzMZJat4AX0VRCAy/xkcOc0=",
    "zh:054b8dd49f0549c9a7cc27d159e45327b7b65cf404da5e5a20da154b90b8a644",
    "zh:0b97bf8d5e03d15d83cc40b0530a1f84b459354939ba6f135a0086c20ebbe6b2",
    "zh:1589a2266af699cbd5d80737a0fe02e54ec9cf2ca54e7e00ac51c7359056f274",
    "zh:6330766f1d85f01ae6ea90d1b214b8b74cc8c1badc4696b165b36ddd4cc15f7b",
    "zh:7c8c2e30d8e55291b86fcb64bdf6c25489d538688545eb48fd74ad622e5d3862",
    "zh:99b1003bd9bd32ee323544da897148f46a527f622dc3971af63ea3e251596342",
    "zh:9b12af85486a96aedd8d7984b0ff811a4b42e3d88dad1a3fb4c0b580d04fa425",
    "zh:9f8b909d3ec50ade83c8062290378b1ec553edef6a447c56dad01a99f4eaa93",
    "zh:aaef921ff9aabaj8b1869a86d692ebd24fbd4e12c21205034bb679b9caf883a2",
    "zh:ac882313207aba00dd5a76dbd572a0ddc818bb9cbf5c9d61b28fe30efaec951e",
    "zh:bb64e8aff37becab373a1a0cc1080990785304141af42ed6aa3dd4913b000421",
    "zh:dfe495f6621df5540d9c92ad40b8067376350b005c637ea6efac5dc15028add4",
    "zh:f0ddf0eaf052766cfe09dea8200a946519f653c384ab4336e2a4a64fdd6310e9",
    "zh:f1b7e684f4c7ae1eed272b6de7d2049bb87a0275cb04dbb7cda6636f600699c9",
    "zh:ff461571e3f233699bf690db319dfe46aec75e58726636a0d97dd9ac6e32fb70",
  ]
}
ubuntu@ip-172-31-79-174:~/terraform-aws-infra$ █

```

Task Question

Difference

~> 5.0

Means

>=5.0

<6.0

Terraform can install

5.1

5.10

5.99

Not

6.0

>=5.0

Means

Any version above 5.0

Even

6

7

8

=5.0.0

Exactly

5.0.0

Nothing else.

Step 7 - Write main.tf

Open

nano main.tf

Paste this complete code.

```
#####  
# VPC  
#####
```

```
resource "aws_vpc" "main" {
```

```
    cidr_block = "10.0.0.0/16"
```

```
    tags = {
```

```
        Name = "TerraWeek-VPC"
```

```

    }

}

#####
# Public Subnet
#####

resource "aws_subnet" "public" {

    vpc_id = aws_vpc.main.id

    cidr_block = "10.0.1.0/24"

    map_public_ip_on_launch = true

    tags = {

        Name = "TerraWeek-Public-Subnet"

    }

}

#####
# Internet Gateway
#####

resource "aws_internet_gateway" "igw" {

    vpc_id = aws_vpc.main.id

    tags = {

        Name = "TerraWeek-IGW"

    }

}

#####
# Route Table
#####

resource "aws_route_table" "public_rt" {

    vpc_id = aws_vpc.main.id

```

```
route {

    cidr_block = "0.0.0.0/0"

    gateway_id = aws_internet_gateway.igw.id

}

}

#####
# Route Table Association
#####

resource "aws_route_table_association" "public_assoc" {

    subnet_id = aws_subnet.public.id

    route_table_id = aws_route_table.public_rt.id

}
```

Save.

Step 8 - Format

```
terraform fmt
```

Step 9 - Validate

```
terraform validate
```

Expected

Success!

```
ubuntu@ip-172-31-79-174:~/terraform-aws-infra$ terraform validate
Success! The configuration is valid.
```

Step 10 - Plan

terraform plan

Should show

5 to add

Resources

VPC

Subnet

IGW

Route Table

Association

```
# aws_vpc.main will be created
+ resource "aws_vpc" "main" {
  + arn                                = (known after apply)
  + cidr_block                        = "10.0.0.0/16"
  + default_network_acl_id           = (known after apply)
  + default_route_table_id           = (known after apply)
  + default_security_group_id        = (known after apply)
  + dhcp_options_id                  = (known after apply)
  + enable_dns_hostnames              = (known after apply)
  + enable_dns_support                = true
  + enable_network_address_usage_metrics = (known after apply)
  + id                               = (known after apply)
  + instance_tenancy                  = "default"
  + ipv6_association_id               = (known after apply)
  + ipv6_cidr_block                   = (known after apply)
  + ipv6_cidr_block_network_border_group = (known after apply)
  + main_route_table_id               = (known after apply)
  + owner_id                          = (known after apply)
  + tags                              = {
    + "Name" = "TerraWeek-VPC"
  }
  + tags_all                          = {
    + "Name" = "TerraWeek-VPC"
  }
}
```

Plan: 5 to add, 0 to change, 0 to destroy.

Step 11 - Apply

terraform apply

Type

yes

Wait.

```
Plan: 5 to add, 0 to change, 0 to destroy.

Do you want to perform these actions?
  Terraform will perform the actions described above.
  Only 'yes' will be accepted to approve.

  Enter a value: yes

aws_vpc.main: Creating...
aws_vpc.main: Creation complete after 1s [id=vpc-091cc994a8d8b5be9]
aws_internet_gateway.igw: Creating...
aws_subnet.public: Creating...
aws_internet_gateway.igw: Creation complete after 0s [id=igw-03dfec4e831b1b047]
aws_route_table.public_rt: Creating...
aws_subnet.public: Still creating... [00m10s elapsed]
aws_route_table.public_rt: Still creating... [00m10s elapsed]
aws_subnet.public: Creation complete after 11s [id=subnet-00553dd0cad06b20f]
aws_route_table.public_rt: Still creating... [00m20s elapsed]
aws_route_table.public_rt: Creation complete after 20s [id=rtb-073a0811c28673e84]
aws_route_table_association.public_assoc: Creating...
aws_route_table_association.public_assoc: Creation complete after 1s [id=rtbassoc-0de0e39ca5a3048f1]

Apply complete! Resources: 5 added, 0 changed, 0 destroyed.
```

Step 12 - Verify AWS Console

Go to

AWS



VPC

Check



VPC



Subnets



Internet Gateway



Route Table

Everything should be connected.



Step 13 - Add Security Group

Add below previous code.

```
resource "aws_security_group" "main" {

  name = "TerraWeek-SG"

  vpc_id = aws_vpc.main.id

  ingress {

    from_port = 22

    to_port = 22

    protocol = "tcp"

    cidr_blocks = ["0.0.0.0/0"]

  }

  ingress {

    from_port = 80

    to_port = 80

    protocol = "tcp"
```

```
    cidr_blocks = ["0.0.0.0/0"]
  }

  egress {

    from_port = 0

    to_port = 0

    protocol = "-1"

    cidr_blocks = ["0.0.0.0/0"]
  }

  tags = {

    Name = "TerraWeek-SG"
  }
}
```

Step 14 - Find Amazon Linux 2 AMI

Run

```
aws ssm get-parameters \
--names /aws/service/ami-amazon-linux-latest/amzn2-ami-hvm-x86_64-gp2 \
--region us-east-1
```

Copy the Value.

Looks like

```
ami-xxxxxxxxxxxxx
```

Step 15 - Create EC2

Add

```
resource "aws_instance" "main" {
```

```
ami = "ami-0b6d9d3d33ba97d99"

instance_type = "t2.micro"

subnet_id = aws_subnet.public.id

vpc_security_group_ids = [

    aws_security_group.main.id

]

associate_public_ip_address = true

tags = {

    Name = "TerraWeek-Server"

}

}

Replace

YOUR_AMI_ID

with your actual AMI.
```

Step 16 - Apply Again

terraform apply

```
Plan: 2 to add, 0 to change, 0 to destroy.

Do you want to perform these actions?
  Terraform will perform the actions described above.
  Only 'yes' will be accepted to approve.

  Enter a value: yes

aws_security_group.main: Creating...
aws_security_group.main: Creation complete after 2s [id=sg-0050394a91b3e9478]
aws_instance.main: Creating...
aws_instance.main: Still creating... [00m10s elapsed]
aws_instance.main: Creation complete after 13s [id=i-02bf02a5769c9ce14]

Apply complete! Resources: 2 added, 0 changed, 0 destroyed.
```

Type

yes

Go to

EC2 Console

Check

 Instance Running

Public IP assigned

Step 17 - Add S3 Bucket

```
resource "aws_s3_bucket" "logs" {  
  
  bucket = "terraform-logs-2026"  
  
  depends_on = [  
  
    aws_instance.main  
  
  ]  
  
}
```

Bucket name must be globally unique. If it already exists, change it (for example, add a random suffix).

Step 18 - Plan

terraform plan

Observe

Terraform waits for EC2 before creating S3 because of

Depends_on

```

ubuntu@ip-172-31-79-174:~/terraform-aws-infra$ terraform plan
aws_vpc.main: Refreshing state... [id=vpc-091cc994a8d8b5be9]
aws_subnet.public: Refreshing state... [id=subnet-00553dd0cad06b20f]
aws_security_group.main: Refreshing state... [id=sg-0050394a91b3e9478]
aws_internet_gateway.igw: Refreshing state... [id=igw-03dfec4e831b1b047]
aws_route_table.public_rt: Refreshing state... [id=rtb-073a0811c28673e84]
aws_instance.main: Refreshing state... [id=i-02bf02a5769c9ce14]
aws_route_table_association.public_assoc: Refreshing state... [id=rtbassoc-0de0e39ca5a3048f1]

Terraform used the selected providers to generate the following execution plan. Resource actions are indicated with the following symbols:
+ create

Terraform will perform the following actions:

# aws_s3_bucket.logs will be created
+ resource "aws_s3_bucket" "logs" {
  + acceleration_status = (known after apply)
  + acl                 = (known after apply)
  + arn                 = (known after apply)
  + bucket              = "terraform-logs-2026"
  + bucket_domain_name = (known after apply)
  + bucket_prefix       = (known after apply)
  + bucket_regional_domain_name = (known after apply)
  + force_destroy       = false
  + hosted_zone_id      = (known after apply)
  + id                  = (known after apply)
  + object_lock_enabled = (known after apply)
  + policy              = (known after apply)
  + region              = (known after apply)
  + request_payer       = (known after apply)
  + tags_all            = (known after apply)
  + website_domain      = (known after apply)
  + website_endpoint    = (known after apply)

  + cors_rule (known after apply)
  + grant (known after apply)

```

Step 19 - Dependency Graph

If Graphviz installed

terraform graph | dot -Tpng > graph.png

Otherwise

terraform graph

```

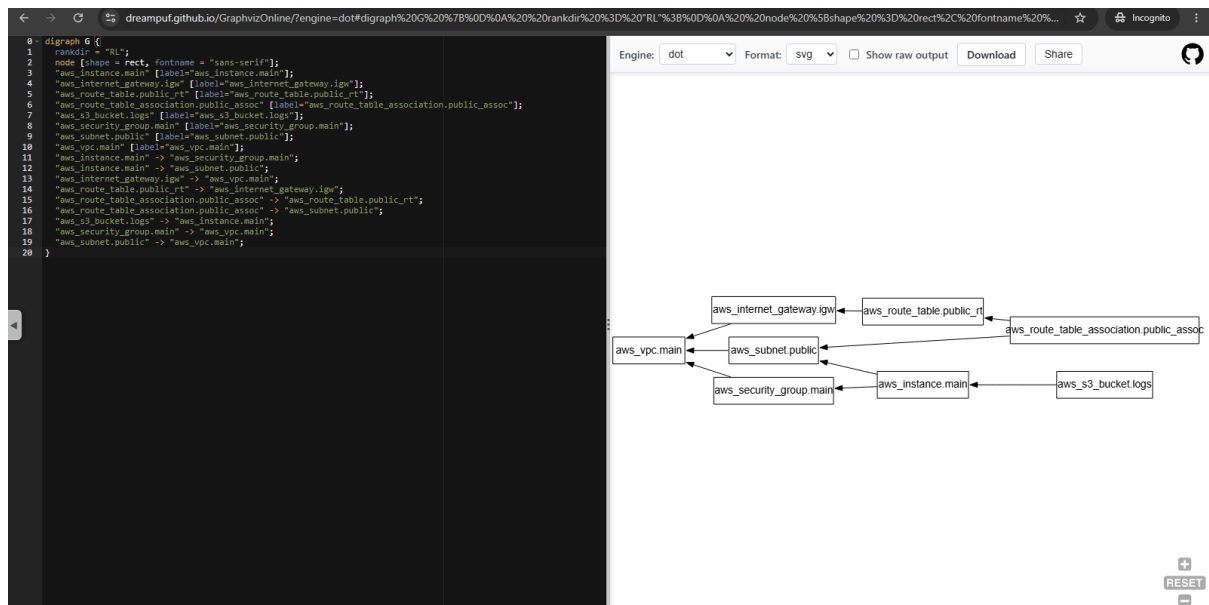
ubuntu@ip-172-31-79-174:~/terraform-aws-infra$ terraform graph
digraph G {
  rankdir = "RL";
  node [shape = rect, fontname = "sans-serif"];
  "aws_instance.main" [label="aws_instance.main"];
  "aws_internet_gateway.igw" [label="aws_internet_gateway.igw"];
  "aws_route_table.public_rt" [label="aws_route_table.public_rt"];
  "aws_route_table_association.public_assoc" [label="aws_route_table_association.public_assoc"];
  "aws_s3_bucket.logs" [label="aws_s3_bucket.logs"];
  "aws_security_group.main" [label="aws_security_group.main"];
  "aws_subnet.public" [label="aws_subnet.public"];
  "aws_vpc.main" [label="aws_vpc.main"];
  "aws_instance.main" -> "aws_security_group.main";
  "aws_instance.main" -> "aws_subnet.public";
  "aws_internet_gateway.igw" -> "aws_vpc.main";
  "aws_route_table.public_rt" -> "aws_internet_gateway.igw";
  "aws_route_table_association.public_assoc" -> "aws_route_table.public_rt";
  "aws_route_table_association.public_assoc" -> "aws_subnet.public";
  "aws_s3_bucket.logs" -> "aws_instance.main";
  "aws_security_group.main" -> "aws_vpc.main";
  "aws_subnet.public" -> "aws_vpc.main";
}
ubuntu@ip-172-31-79-174:~/terraform-aws-infra$ █

```

Copy output.

Go to

<https://dreampuf.github.io/GraphvizOnline/>



Paste it.

Download image.

[main.tf](#)-----fullcode till now

#####

VPC

#####

resource "aws_vpc" "main" {

cidr_block = "10.0.0.0/16"

tags = {

Name = "TerraWeek-VPC"

}

}

#####

Public Subnet

#####

resource "aws_subnet" "public" {

vpc_id = aws_vpc.main.id

cidr_block = "10.0.1.0/24"


```

map_public_ip_on_launch = true

tags = {

    Name = "TerraWeek-Public-Subnet"

}

}

#####
# Internet Gateway
#####

resource "aws_internet_gateway" "igw" {

    vpc_id = aws_vpc.main.id

    tags = {

        Name = "TerraWeek-IGW"

    }

}

#####
# Route Table
#####

resource "aws_route_table" "public_rt" {

    vpc_id = aws_vpc.main.id

    route {

        cidr_block = "0.0.0.0/0"

        gateway_id = aws_internet_gateway.igw.id

    }

}

#####
# Route Table Association
#####

```

```
resource "aws_route_table_association" "public_assoc" {

  subnet_id = aws_subnet.public.id

  route_table_id = aws_route_table.public_rt.id
}
resource "aws_security_group" "main" {

  name = "TerraWeek-SG"

  vpc_id = aws_vpc.main.id

  ingress {

    from_port = 22

    to_port = 22

    protocol = "tcp"

    cidr_blocks = ["0.0.0.0/0"]
  }

  ingress {

    from_port = 80

    to_port = 80

    protocol = "tcp"

    cidr_blocks = ["0.0.0.0/0"]
  }

  egress {

    from_port = 0

    to_port = 0

    protocol = "-1"

    cidr_blocks = ["0.0.0.0/0"]
  }
}
```

```
}

tags = {

    Name = "TerraWeek-SG"

}

}

resource "aws_instance" "main" {

    ami = "ami-0b6d9d3d33ba97d99"

    instance_type = "t2.micro"

    subnet_id = aws_subnet.public.id

    vpc_security_group_ids = [

        aws_security_group.main.id

    ]

    associate_public_ip_address = true

    tags = {

        Name = "TerraWeek-Server"

    }

}

resource "aws_s3_bucket" "logs" {

    bucket = "terraform-logs-2026"

    depends_on = [

        aws_instance.main

    ]

}
```

Step 20 - Lifecycle Rule

Inside EC2 resource

(In this i pasted this line add lifecycle and changed ami no - i choice
ami-0a29987e14ae814db-t2micro)

Add

```
lifecycle {
```

```
    create_before_destroy = true
```

```
}
```

Now change AMI ID.

Run

terraform plan

Terraform creates

New Instance

↓

Old Instance Destroy

instead of downtime.

```
Terraform will perform the following actions:

# aws_instance.main must be replaced
-/- resource "aws_instance" "main" {
  - ami                               = "ami-0b6d9d3d33ba97d99" -> "ami-091b599f5f318ddd2" & forces replacement
  - arm                               = "arn:aws:ec2:us-east-1:769369611777:instance/i-02bf02a5769c9ce14" -> (known after apply)
  - availability_zone                 = "us-east-1f" -> (known after apply)
  - cpu_core_count                    = 1 -> (known after apply)
  - cpu_threads_per_core              = 1 -> (known after apply)
  - disable_api_stop                   = false -> (known after apply)
  - disable_api_termination            = false -> (known after apply)
  - ebs_optimized                     = false -> (known after apply)
  + enable_primary_ipv6                = (known after apply)
  - hibernation                       = false -> null
  + host_id                           = (known after apply)
  + host_resource_group_arn            = (known after apply)
  + iam_instance_profile                = (known after apply)
  - id                                = "i-02bf02a5769c9ce14" -> (known after apply)
  - instance_initiated_shutdown_behavior = "stop" -> (known after apply)
  - instance_lifecycle                 = (known after apply)
  - instance_state                     = "running" -> (known after apply)
  - ipv6_address_count                 = 0 -> (known after apply)
  - ipv6_addresses                     = [] -> (known after apply)
  + key_name                           = (known after apply)
  - monitoring                         = false -> (known after apply)
  - outpost_arn                       = (known after apply)
  + password_data                      = (known after apply)
  + placement_group                    = (known after apply)
  - placement_partition_number         = 0 -> (known after apply)
  - primary_network_interface_id       = "eni-088217e7705ba9933" -> (known after apply)
  - private_dns                        = "ip-10-0-1-47.ec2.internal" -> (known after apply)
  - private_ip                         = "10.0.1.47" -> (known after apply)
  - public_dns                         = (known after apply)
  - public_ip                         = "44.213.77.100" -> (known after apply)
  - secondary_private_ips               = [] -> (known after apply)
```

```
Plan: 2 to add, 0 to change, 1 to destroy.
```

Terraform apply

```
Plan: 2 to add, 0 to change, 1 to destroy.

Do you want to perform these actions?
  Terraform will perform the actions described above.
  Only 'yes' will be accepted to approve.

  Enter a value: yes

aws_instance.main: Creating...
aws_instance.main: Still creating... [00m10s elapsed]
aws_instance.main: Creation complete after 13s [id=i-0dc0bd0beda42ebbf]
aws_s3_bucket.logs: Creating...
aws_s3_bucket.logs: Creation complete after 0s [id=terraform-logs-2026]
aws_instance.main (deposed object 78fd33d7): Destroying... [id=i-02bf02a5769c9ce14]
aws_instance.main: Still destroying... [id=i-02bf02a5769c9ce14, 00m10s elapsed]
aws_instance.main: Still destroying... [id=i-02bf02a5769c9ce14, 00m20s elapsed]
aws_instance.main: Still destroying... [id=i-02bf02a5769c9ce14, 00m30s elapsed]
aws_instance.main: Still destroying... [id=i-02bf02a5769c9ce14, 00m40s elapsed]
aws_instance.main: Still destroying... [id=i-02bf02a5769c9ce14, 00m50s elapsed]
aws_instance.main: Still destroying... [id=i-02bf02a5769c9ce14, 01m00s elapsed]
aws_instance.main: Still destroying... [id=i-02bf02a5769c9ce14, 01m10s elapsed]
aws_instance.main: Destruction complete after 1m10s

Apply complete! Resources: 2 added, 0 changed, 1 destroyed.
```

Step 21 - Destroy

terraform destroy

Type

yes

Watch order.

Terraform destroys

EC2

↓

Security Group

↓

Association

↓

Route Table

↓

Internet Gateway

↓

Subnet

↓

VPC

(reverse dependency order)

```
Destroy complete! Resources: 8 destroyed.
```

Task Answers

How does Terraform know to create the VPC first?

The subnet resource references `aws_vpc.main.id`. Terraform builds a dependency graph from these references and automatically creates the VPC before any resource that depends on it.

What happens if the subnet is created before the VPC?

AWS rejects the request because the referenced VPC does not exist. Terraform prevents this by respecting the dependency graph.

Implicit Dependencies

- `aws_subnet.public` → `aws_vpc.main`
- `aws_internet_gateway.igw` → `aws_vpc.main`
- `aws_route_table.public_rt` → `aws_vpc.main`
- `aws_route_table.public_rt` → `aws_internet_gateway.igw`
- `aws_route_table_association.public_assoc` → `aws_subnet.public`
- `aws_route_table_association.public_assoc` → `aws_route_table.public_rt`

- `aws_security_group.main → aws_vpc.main`
- `aws_instance.main → aws_subnet.public`
- `aws_instance.main → aws_security_group.main`

When to use `depends_on`?

Use it when Terraform cannot infer the dependency automatically. Examples:

- Create an S3 bucket only after an EC2 instance is provisioned for application setup.
- Ensure a monitoring or configuration resource is created only after the underlying infrastructure is fully available.

Lifecycle Arguments

Argument	Purpose	Example Use
<code>create_before_destroy</code>	Creates a replacement resource before deleting the old one to reduce downtime	Replacing an EC2 instance during an AMI update
<code>prevent_destroy</code>	Blocks accidental deletion of a resource	Production databases or critical S3 buckets
<code>ignore_changes</code>	Ignores changes to selected attributes outside Terraform	Ignore tag updates or auto-scaled desired capacity managed elsewhere

After completing the lab, take screenshots of:

- `terraform apply` output
- AWS VPC console showing the VPC, subnet, IGW, and route table
- EC2 instance with its public IP
- Dependency graph (`graph.png` or Graphviz output)

Then create your `day-62-providers-resources.md` with your commented `main.tf`, screenshots, dependency graph, and the explanations above.